Sumário

1. Motivação e Objetivos
2. Introdução
3. Histórico e Evolução das Arquiteturas Paralelas
4. Visão Geral da CAD
   - 4.1. Precisamos Escrever Códigos Paralelos?
   - 4.2. Quais Aplicações Necessitam da CAD?
   - 4.3. Onde a CAD é realizada?
   - 4.4. A CAD tem futuro?
5. Conceitos Básicos da CAD
   - 5.1. Sistemas Multiprocessados;
   - 5.2. Sistemas *Multicore*;
   - 5.3. Sistemas *Manycore*;
6. Ferramentas Paralelas
   - 6.1. OpenMP
   - 6.2. MPI
   - 6.3. OpenACC
   - 6.4. CUDA

Conceitos Básicos da CAD



- Como mencionado no slide 44...

- A CAD pode ser realizada/desenvolvida através de arquiteturas como:
  - ✓ Uso de Máquinas Vetoriais;
  - ✓ Sistemas Multiprocessados;
  - ✓ Sistemas *Multicore*;
  - ✓ Sistemas *Manycore*;

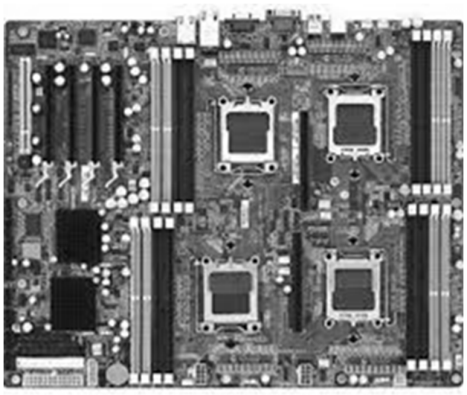
Conceitos Básicos da CAD

- Arquitetura Vetorial
  - É um modelo computacional projetado para processar eficientemente grandes conjuntos de dados usando uma única instrução;
  - Esse modelo segue o princípio SIMD (*Single Instruction, Multiple Data*);

Característica	Arq. Escalar	Arq. Vetorial
Operação	Executa uma operação por instrução.	Executa a mesma operação em vários dados por instrução
Controle de Loop	Requer um laço de repetição no código para iterar sobre o array	Elimina o laço. A instrução opera sobre o array inteiro de uma vez.
Exemplo	C[i] = A[i] + B[i]	C = A +B

Conceitos Básicos da CAD

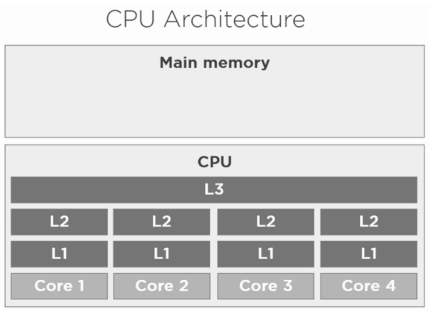
- Sistemas Multiprocessados
  - Sistemas dessa natureza podem ser implementados, tanto em memória compartilhada, quanto em memória distribuída.

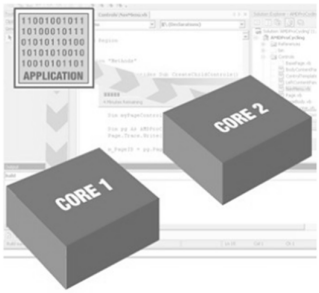




Conceitos Básicos da CAD

- Sistemas *Multicore*
  - Arquitetura na qual um único processador físico contém duas ou mais unidades de execução (*cores*);
  - A finalidade é criar um sistema que possa executar mais tarefas, em intervalos de tempos cada vez menores (melhorando o desempenho do sistema);





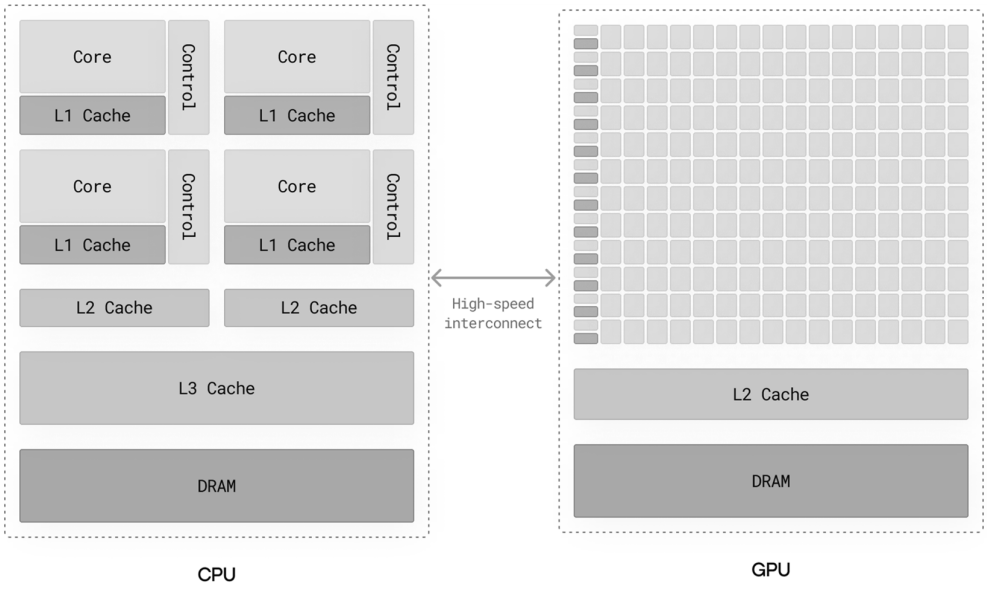
  - Sistema operacional trata como se cada núcleo fosse um processador diferente, com seus próprios recursos;

Conceitos Básicos da CAD

- Sistemas *Multicore*
  - Vantagens:
    - ✓ Melhora o paralelismo no nível de *threads*;
    - ✓ Maior eficácia (*throughput*) do sistema e desempenho aprimorado de aplicativos em computadores, pois pode executar vários simultaneamente;
    - ✓ Economia de espaço e de energia;
    - ✓ Melhor comunicação entre as unidades;

Conceitos Básicos da CAD

- Sistemas *Manycore*
  - ✓ Possui uma quantidade muito maior de *cores* independentes, quando comparados aos processadores *multicore*;



Conceitos Básicos da CAD

- Sistemas *Manycore*
  - ✓ Exemplo:



- NVIDIA GeFORCE RTX 5090
- Núcleos: 21.760
- Frequência: 1.189 MHz
- Memória Máxima: 32 GB GDDR7
- Largura de Banda: 1.792 GB/s
- Preço: $ 2.000,00

Sumário

1. Motivação e Objetivos
2. Introdução
3. Histórico e Evolução das Arquiteturas Paralelas
4. Visão Geral da CAD
   - 4.1. Precisamos Escrever Códigos Paralelos?
   - 4.2. Quais Aplicações Necessitam da CAD?
   - 4.3. Onde a CAD é realizada?
   - 4.4. A CAD tem futuro?
5. Conceitos Básicos da CAD
   - 5.1. Sistemas Multiprocessados;
   - 5.2. Sistemas *Multicore*;
   - 5.3. Sistemas *Manycore*;
6. Ferramentas Paralelas
   - 6.1. OpenMP
   - 6.2. MPI
   - 6.3. OpenACC
   - 6.4. CUDA

Ferramentas Paralelas

- Ao mesmo tempo que há um progresso no desenvolvimento de hardware, há uma necessidade de se oferecer recursos de programação compatíveis e padronizados com os diferentes ambientes computacionais disponíveis;
- Existem várias bibliotecas/ferramentas para programação paralela padronizada e eficiente;
- Tanto para ambientes de memória compartilhada, quanto para memória distribuída.
  - ✓ OpenMP (*Open Multi-Processing*);
  - ✓ MPI (*Message Passing Interface*);
  - ✓ OpenACC (*Open Accelerators*);
  - ✓ CUDA (*Compute Unified Device Architecture*)

Ferramentas Paralelas

OpenMP

- OpenMP (*Open Multi-Processing*)
  - Histórico:
    - ✓ Foram publicadas as primeiras especificações da API, para Fortran, em outubro de 1997;
    - ✓ Em outubro de 1998, lançaram o padrão para as linguagens C e C++;
    - ✓ Em 2005, foi lançada uma versão combinada para C, C++ e Fortran;
    - ✓ Atualmente, está na versão 6.0, lançada em Novembro de 2024;
    - ✓ Site: https://www.openmp.org/

Ferramentas Paralelas

OpenMP

- OpenMP (*Open Multi-Processing*)
  - Objetivos:
    - ✓ Foi projetado para a programação de computadores paralelos com memória compartilhada;
    - ✓ Existência de um único espaço de endereçamento através de todo o sistema de memória (principal facilidade);
    - ✓ Cada unidade de processamento pode ler e escrever em todas as posições de memória;
    - ✓ O paralelismo no OpenMP é obtido pela execução simultânea de diversas *threads* dentro das regiões paralelas;

Ferramentas Paralelas

- OpenMP (*Open Multi-Processing*)
  - Objetivos:
    - ✓ O OpenMP faz uso combinado de diretivas passadas para o compilador, assim como de funções definidas na sua biblioteca;
  - Novidades:
    - ✓ O OpenMP ARB formou um subcomitê da Linguagem Python para definir como as **diretivas** OpenMP podem ser suportadas em Python.



Ferramentas Paralelas

- OpenMP (*Open Multi-Processing*)
  - Exemplos:

Programa sequencial

```
void main()
{
  double r[1000];

  for (int i=0; i<1000; i++) {
    large_computation(r[i]);
  }
}
```

Programa paralelo

```
void main()
{
  double r[1000];

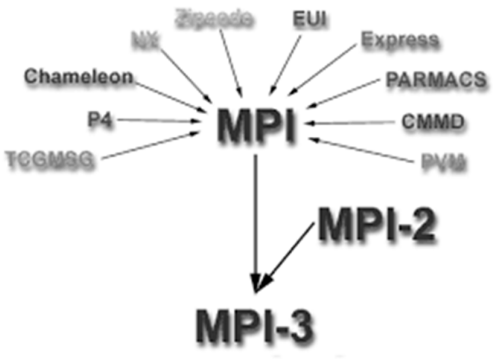
#pragma omp parallel for
  for (int i=0; i<1000; i++) {
    large_computation(r[i]);
  }
}
```

Ferramentas Paralelas



Open MPI:
Open Source High Performance Computing

- MPI (*Message Passing Interface*)
  - Histórico:
    - ✓ Surgiu por um esforço de padronização de uma ferramenta paralela de um ambiente de memória distribuída para a portabilidade e facilidade de uso;



- ✓ Esse processo de padronização teve início com o *Workshop on Standards for Message Passing in a Distributed Memory Environment*, em abril de 1992;
- ✓ A primeira versão (1.0) foi apresentada na conferência "*Supercomputing*" em novembro de 1993;
- ✓ Atualmente, está na versão 5.0.10, lançada em Fev/2026;
- ✓ https://www.open-mpi.org/

Ferramentas Paralelas



Open MPI:
Open Source High Performance Computing

- MPI (*Message Passing Interface*)
  - Objetivos:
    - ✓ Oferecer uma interface padrão para o desenvolvimento de programas baseados no paradigma de troca de mensagens;
    - ✓ Essa interface pode ser implementada em diversas plataformas, sem mudanças significativas no sistema de comunicação subjacente ou software de sistema;
    - ✓ Garante uma comunicação eficiente:
      - Permitindo superposição de comunicação e computação;
      - Comunicação de processo a processo e em grupo;

Ferramentas Paralelas



Open MPI:
Open Source High Performance Computing

- MPI (*Message Passing Interface*)
  - Exemplo:

```
int main(){
    printf("Hello World");
    return 0;
}
```

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv){
   int rank, size;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   printf("Hello World! I'm %d of %d\n", rank,
size);

   MPI_Finalize();
   return 0;
}
```

Ferramentas Paralelas



Open MPI:
Open Source High Performance Computing

- MPI (*Message Passing Interface*)
  - Exemplo:

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char **argv){
   int rank, size, tag, i;
   MPI_Status status;
   char msg[20];
   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);
   if(rank == 0){
      strcpy(msg, "Hello World!\n");
      for(i=1; i < size; i++)
         MPI_Send(msg, 13, MPI_CHAR, i, tag, MPI_COMM_WORLD);
   }else{
      MPI_Recv(msg, 20, MPI_CHAR, 0,tag, MPI_COMM_WORLD,
&status);
      printf("Process %d: Message received: %s\n", rank,
msg);
   }
   MPI_Finalize();
   return 0;
}
```

Ferramentas Paralelas

- OpenACC (*Open Accelerators*)
  - ✓ Foi projetado para que os códigos pudessem ser portados em uma ampla variedade de plataformas e arquiteturas de hardware heterogêneas;
  - ✓ É um modelo de programação paralela portátil, baseada em diretivas definidas pelo usuário;
  - ✓ Requer um "esforço" muito menor de programação do que os modelos de baixo nível;
  - ✓ Fornece suporte às linguagens de programação C, C++ e Fortran;
    - Além dos aceleradores baseados em GPU da NVIDIA e AMD;

Ferramentas Paralelas

- OpenACC (*Open Accelerators*)
  - ❑ Histórico:
    - ✓ A primeira versão (1.0) foi lançada em Nov/2011;
    - ✓ A versão 2.0 foi lançada em Jun/2013 e teve uma atualização em Ago/2013;
    - ✓ Atualmente, está na versão 3.3, lançada em Nov/2022;
    - ✓ Site: https://www.openacc.org/

Ferramentas Paralelas

OpenACC
More Science, Less Programming

- OpenACC (*Open Accelerators*)
  - Objetivos:
    - ✓ Fácil e simples de utilizar, sendo independente de fabricante;
    - ✓ Oculta a complexidade de hardware dos programadores;
    - ✓ Requer pouca modificação no código fonte;
    - ✓ Pode ser combinado com os modelos de programação MPI e OpenMP;
    - ✓ O mesmo código, sem modificações, pode ser usado em diferentes arquiteturas;

Ferramentas Paralelas

OpenACC
More Science, Less Programming

- OpenACC (*Open Accelerators*)
  - Exemplo:

```
#include <stdio.h>
#define N 1000000000
int main(void) {
      double pi = 0.0f; long i;


      for (i=0; i<N; i++)
      {
            double t=(double) ((i+0.5)/N);
            pi += 4.0/(1.0+t*t);
      }
      printf("pi=%f\n",pi/N);
      return 0;
}
```

```
#include <stdio.h>
#define N 1000000000
int main(void) {
      double pi = 0.0f; long i;

      #pragma acc   parallel loop reduction(+: pi)
      for (i=0; i<N; i++)
      {
            double t=(double) ((i+0.5)/N);
            pi += 4.0/(1.0+t*t);
      }
      printf("pi=%f\n",pi/N);
      return 0;
}
```

Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)
  - Histórico:
    - ✓ Os aceleradores gráficos surgiram com uma necessidade de se melhorar o desempenho de aplicações gráficas, como jogos eletrônicos;
    - ✓ Com a evolução dos jogos 3D foi necessário evoluir, também, o poder computacional;
    - ✓ As placas gráficas passaram a ser cada vez mais poderosas;



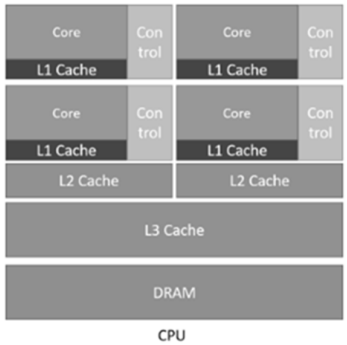


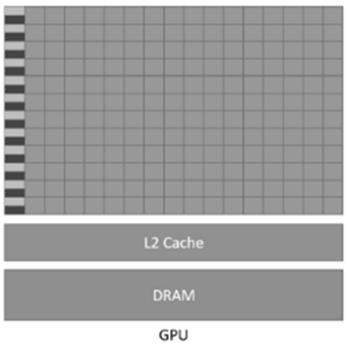
Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)
  - Histórico:

- ✓ A estruturas das placas gráficas começou chamar a atenção dos pesquisadores, pois quando comparadas as CPUs, as placas gráficas possuíam um número bem maior de *cores;*
- ✓ Esses *cores* poderiam ser utilizados para processar, também, **dados de uso geral.**





Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)
  - Histórico:
    - ✓ Até 2006, as placas gráficas eram muito difíceis de usar, pois os programadores tinham que usar funções da API gráfica para acessar os núcleos dos processadores (técnicas de OpenGL e Direct3D eram necessárias);
    - ✓ Essa técnica era chamada GPGPU (*General-Purpose Graphics Processing Unit*);
    - ✓ As APIs limitavam os tipos de aplicações que se poderiam ter para esses tipos de placas;
    - ✓ Poucos usuários e grandes habilidades para se conseguir um bom desempenho;

Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)
  - Histórico:

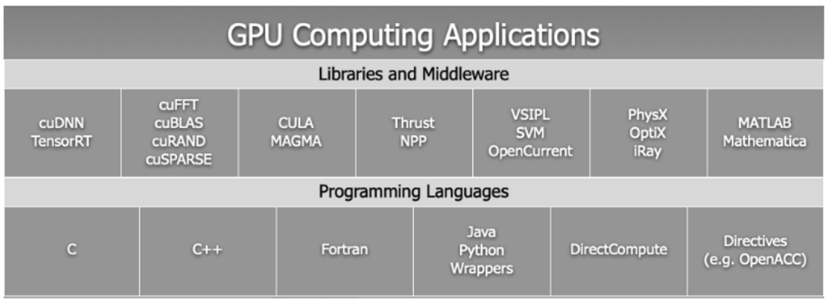


- ✓ Em Nov/2006, tudo mudou !!!
  - A NVIDIA lançou o CUDA e os programas não passam mais pela interface gráfica;
  - Uma nova interface de programação paralela, de uso geral (não apenas voltada para aplicações gráficas) na placa, atende as solicitações dos programas CUDA;
  - Camadas de softwares foram refeitas de modo que os programadores poderiam programar utilizando ferramentas de programação C/C++ tradicionais;
- ✓ Site: https://developer.nvidia.com/cuda-downloads

Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)
  - Objetivos/Vantagens:
    - ✓ Proporcionar ganhos de desempenho maiores que o uso de CPUs, por possuir dezenas e/ou centenas de núcleos;
    - ✓ Baixa curva de aprendizado para programadores familiarizados com linguagens de programação padrão;
    - ✓ As GPUs estão presentes em todos os grandes supercomputadores;



Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)

  - Exemplo:

```c
#include <stdio.h>
#include <stdlib.h>
#include <iostream>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <iostream>

__global__ void kernel(void){
    printf("Hello, World from GPU!");
}

int main(void) {
    kernel<<<1,1>>>();
    printf("Fim");
    return 0;
}
```

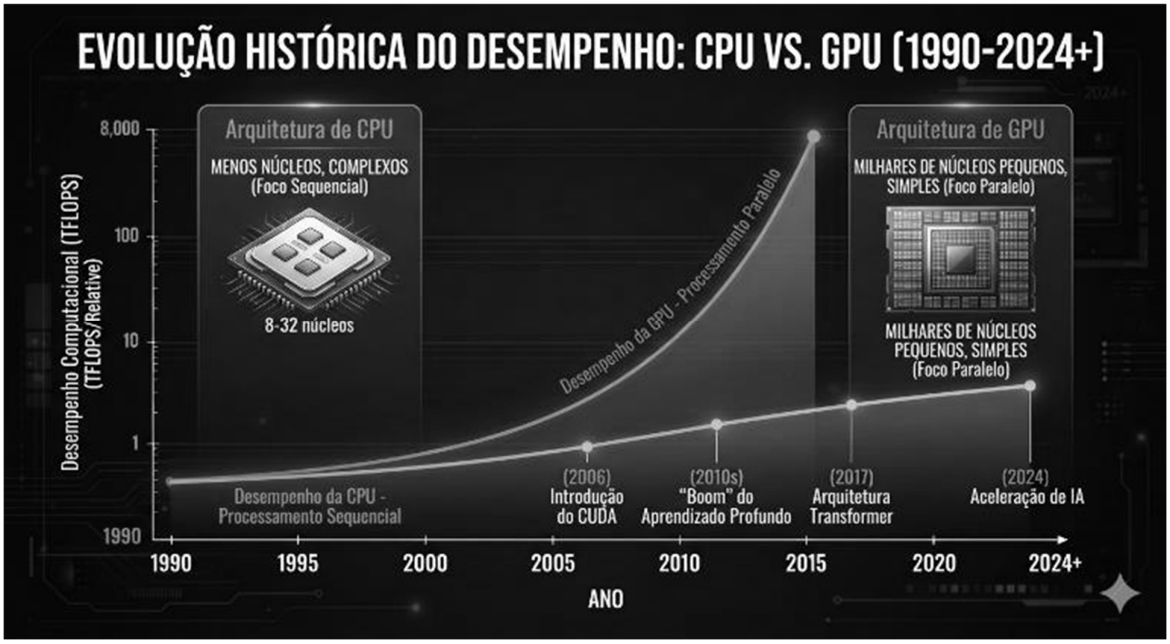
Alerta o compilador que a função deve ser compilada para executar na GPU

Função de kernel: dimensões (blocos/grid)

Ferramentas Paralelas

CUDA Toolkit

- CUDA (*Compute Unified Device Architecture*)
  - Evolução de Desempenho: CPU x GPU



Conclusão

- A Computação de Alto Desempenho é uma realidade na solução de problemas das mais variadas áreas;
- Evoluiu naturalmente com as tecnologias de hardware, software e rede;
- A Programação Paralela é uma necessidade devido aos inúmeros problemas que podem ser solucionados;
- Várias ferramentas evoluíram com a necessidade de padronizar as formas de se programar de modo paralelo;
- Essas ferramentas podem ser executadas tanto em ambientes com memória compartilhada quanto distribuída;

Referências

- https://www.top500.org/
- https://www.openmp.org/
- https://www.open-mpi.org/
- https://www.openacc.org/
- https://developer.nvidia.com/cuda-downloads